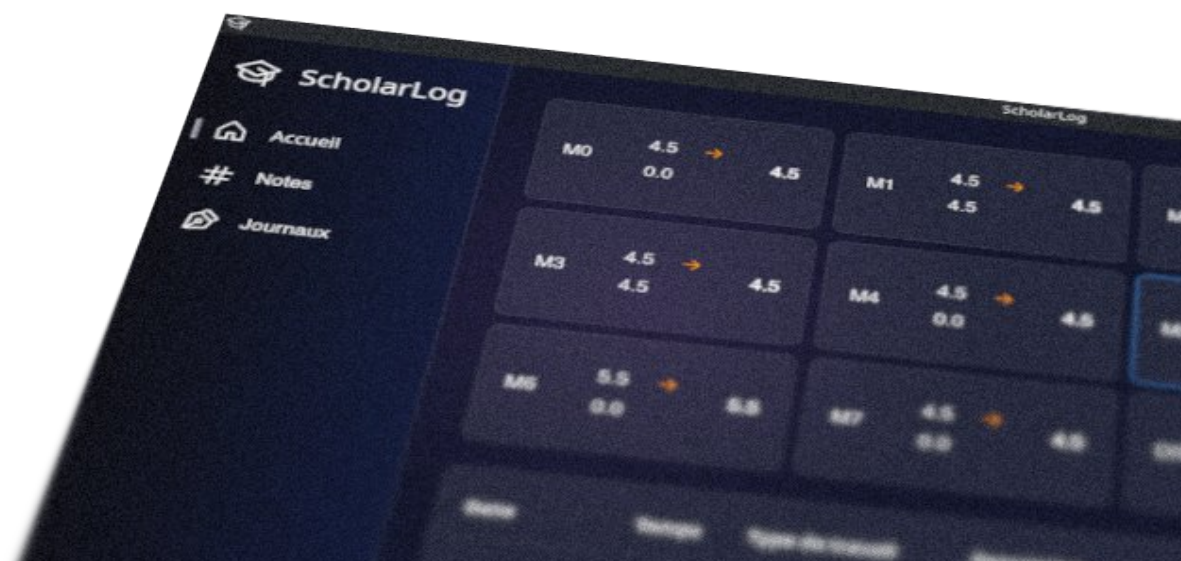




ScholarLog

M5 - Documentation technique

Auteur :
Hofer Lukas
Technicien ES en informatique 2^e année

Responsables du module :
Choffat Lise
Claude Fahrni
Montavon Dominique

Table des matières

1. Introduction.....	5
1.1 Personnes impliquées.....	5
1.2 Directives générales	5
1.2.1 Délais	5
1.2.2 Livrables.....	5
2. Spécifications de l'application.....	6
2.1 Public cible.....	6
2.2 Exigences fonctionnelles	6
2.3 Exigences Non-Fonctionnelles.....	7
2.4 Stockage des données persistantes.....	7
2.4.1 Choix technologique.....	7
2.5 Langage de programmation et Framework d'interface.....	7
2.5.1 Langage : C# (.NET).....	7
2.5.2 Framework Graphique : Avalonia UI	8
2.6 Architecture logicielle.....	8
2.6.1 Architecture interne.....	8
2.7 Diagramme de Cas d'Utilisation	9
2.8 Modèle Conceptuel de Données (MCD)	10
2.9 Interface utilisateur	11
2.9.1 Choix de design.....	11
2.9.2 Pages	11
2.9.2.2 Note.....	12
3. Conception de l'application.....	15
3.1 Architecture globale et Modélisation.....	15
3.2 Accès aux données	15
3.2.1 Le Contexte de Base de Données (MonDbContext)	15
3.2.2 Le dépôt de Données (DataRepository)	16
3.2.3 Le Service Global (AppDataService).....	16
3.3 Le Modèle de Données.....	16
3.3.1 Les énumérations.....	16
3.3.2 Les entités du domaine (Entity Framework Core).....	17
3.3.3 Les Modèles de Vue (ViewModels)	17
3.3.4 Diagramme des entités (Model).....	18

3.4 Implémentation de la page Paramètres.....	19
3.4.1 Gestion de l'apparence (Thème)	19
3.4.2 Gestion de la base de données (Chemin d'accès)	19
3.5 Design et ressources globales.....	19
3.5.1 Gestion des Thèmes (Mode Clair / Mode Sombre)	19
3.5.2 Typographie et Système d'Icônes.....	20
3.5.3 Cohérence Visuelle et Dimensions.....	20
3.6 Architecture MVVM et Logique de Présentation.....	21
3.7 Services et Gestion d'État Globale.....	22
3.8 Optimisation et Déploiement de la Base de Données.....	22
3.9 Traitement et Gestion des Fichiers Jointes (PDF/CBZ)	23
3.10 Exportation des données.....	23
3.11 Composants Utilitaires et d'Interface Graphique.....	24
3.11.1 ViewLocator.....	24
3.11.2 WhiteSlideUpTransition.....	24
3.11.3 ObservableRangeCollection	24
3.11.4 Modules de Traitement des Fichiers (ScanHandlers).....	25
3.12 Composants Visuels Personnalisés (UserControls).....	25
3.12.1 DonutChartControl (Graphique de répartition).....	25
3.12.2 ModalExport (Interface d'exportation de données).....	27
3.12.3 PdfViewerControl (Visionneuse PDF et Scans optimisée)	27
3.12.4 LoadingView (Écran d'attente).....	28
3.13 Configuration Globale et Fichiers de Ressources (App.xaml).....	28
3.13.1 Le Dictionnaire de Ressources (App.xaml)	28
4. Tests.....	29
4.1 Configuration & Paramètres.....	29
4.2 Gestion des Modules et Branches (CRUD)	30
4.3 Exportation des données.....	30
4.4 Gestion des Notes.....	31
4.5 Journal de Travail.....	32
4.6 Tableau de bord & Moyennes (Accueil)	33
4.7 Système d'exploitation (OS).....	33
5. Déploiement de l'application.....	34
5.1 Génération des exécutables (dotnet publish)	34

5.1.1 Windows.....	34
5.1.2 Linux.....	35
5.1.3 MacOS.....	35
5.2 Création de l'installateur Windows Universel (Inno Setup)	35
5.3 Création du paquet Debian (.deb) pour Linux.....	36
5.4 Création du paquet Flatpak (Linux Universel)	36
5.5 Script de déploiement totalement automatisé.....	37
6. Licence du logiciel (GPLv3-only).....	38
7. Gestionnaire de version	39
7.2 Outil d'interface : LazyGit.....	39
8. Conclusion.....	40
8.1 objectifs et choix techniques.....	40
8.2 Déploiement et automatisation.....	40
8.3 Perspectives d'évolution	40
9. Retour sur la planification.....	41
10. Annexes et livrables.....	42

1. Introduction

Ce module d'apprentissage a pour objectif de couvrir l'ensemble du cycle de développement d'une application graphique interactive, depuis l'analyse des besoins jusqu'aux phases de test et de validation.

Il met l'accent sur les bonnes pratiques de conception logicielle, l'architecture orientée objet et l'utilisation d'outils de développement professionnels.

Les compétences visées sont les suivantes :

- Analyser les besoins et définir les spécifications d'une application GUI
- Concevoir l'architecture logicielle selon les principes de la programmation orientée objet (OOP)
- Implémenter une interface graphique interactive
- Appliquer une méthodologie de développement basée sur le cycle de vie avec prototypage rapide
- Réaliser les phases de test et de validation fonctionnelle
- Utiliser efficacement un environnement de développement intégré (IDE)
- Produire une documentation utilisateur claire et exploitable

Présenter le logiciel en anglais dans une approche orientée utilisateur (présentation non technique, mise en valeur des fonctionnalités)

1.1 Personnes impliquées

Étudiant impliqué :

- Hofer Lukas : technicien ES en informatique 2^e année

Enseignants impliqués :

- Choffat Lise : évaluation en anglais
- Claude Fahrni : suivi technique
- Montavon Dominique : responsable et suivi technique du projet

1.2 Directives générales

1.2.1 Délais

Présentation	11 Juin 2026
Remise du travail complet	17 Juin 2026

1.2.2 Livrables

- Documentation technique (ce présent document)
- Documentation utilisateur (en anglais)
- Présentation orienté client (en anglais)
- Planification (avec suivi)
- Journal de travail

2. Spécifications de l'application

Développer une application de bureau permettant à un élève de :

- Inscrire et gérer les notes
- calculer automatiquement des moyennes
- assurer un suivi journalisé de son travail (journal de travail)
- conserver les données localement sans dépendance à un serveur distant

L'application est conçue pour un usage individuel, sur ordinateur, avec les données en local.

2.1 Public cible

L'application est spécifiquement conçue pour les techniciens de l'École Supérieure Technique.

Actuellement, les étudiants doivent gérer leur suivi de formation de manière assez artisanale. Ce projet part de deux constats principaux :

- Le calcul des moyennes : Chaque élève calcule ses notes manuellement. C'est non seulement fastidieux, mais cela peut vite mener à des erreurs de calcul ou à un oubli dans le suivi des résultats au fil du semestre.
- Le journal de travail : Il existe bien une feuille Excel à disposition, mais elle est devenue trop complexe à cause d'une utilisation excessive de macros. Ce n'est pas toujours pratique à utiliser et c'est difficile à faire évoluer.

L'idée de cette application est donc de simplifier tout ça en proposant un outil unique, fiable et beaucoup plus simple à prendre en main que le fichier Excel actuel.

2.2 Exigences fonctionnelles

Les fonctionnalités suivantes devront être implémentées

ID	Catégorie	Description de l'exigence	Priorité
REQ-F-001	Données Scolaires	Le système doit permettre la création, la modification et la suppression de Modules et de Cours associés.	Haute
REQ-F-002	Données Scolaires	L'utilisateur doit pouvoir saisir une note et l'associer à un cours existant.	Haute
REQ-F-003	Calculs	Le système doit calculer automatiquement la moyenne pondérée par cours et module.	Haute
REQ-F-004	Journal	Le système doit permettre l'ajout d'entrées journalisées (Date, Description, Durée) pour le suivi du module.	Haute
REQ-F-005	Export	Le système doit offrir une fonctionnalité d'exportation des données au format standardisé CSV ou tableur.	Moyenne
REQ-F-006	Persistance	Le système doit permettre à l'utilisateur de définir l'emplacement de sauvegarde du fichier de données local.	Basse

2.3 Exigences Non-Fonctionnelles

L'application doit être exécutable sans modification majeure sur les systèmes d'exploitation Windows, macOS et Linux.

Le système doit assurer l'intégrité des données locales pour prévenir toute corruption en cas de plantage.

L'interface graphique (GUI) doit être ergonomique et ne pas nécessiter plus de 3 clics pour accéder à une fonction (par ex : Saisie d'une note).

2.4 Stockage des données persistantes

Afin de garantir l'autonomie de l'application (pas de serveur distant) et la portabilité des données entre différents systèmes d'exploitation (Windows, macOS, Linux), le stockage des données sera géré localement sous la forme d'un fichier unique.

2.4.1 Choix technologique

Le format de stockage retenu est une base de données relationnelle locale SQLite.

Ce choix est justifié par les points suivants :

- Intégrité (Réponse aux Exigences Non-Fonctionnelles) : SQLite respecte les propriétés ACID (Atomicité, Cohérence, Isolation, Durabilité), ce qui minimise considérablement le risque de corruption de données en cas de fermeture inopinée de l'application ou de panne système.
- Portabilité : Le fichier de base de données est un fichier binaire unique, facile à copier, déplacer ou sauvegarder par l'étudiant.
- Indépendance : Aucun moteur de base de données serveur (comme MySQL ou PostgreSQL) n'est nécessaire ; le moteur est embarqué directement dans l'application.

2.5 Langage de programmation et Framework d'interface

2.5.1 Langage : C# (.NET)

Le langage retenu pour la logique métier et l'accès aux données est le C#.

Ce choix se justifie par :

Typage fort et Sécurité : C# offre une grande rigueur dans la gestion des données, essentielle pour les calculs de moyennes et la gestion intègre des notes.

- Performance : Le framework .NET assure une exécution rapide des traitements locaux.
- Maintenance : C'est un langage standard dans l'industrie, facilitant la reprise du code ou l'ajout de fonctionnalités futures par d'autres développeurs.
- Connaissance : Le langage étant enseigné à la division technique, c'est un langage maîtrisé.

2.5.2 Framework Graphique : Avalonia UI

Pour l'interface utilisateur, le choix s'est porté sur Avalonia UI. Contrairement à un framework traditionnels comme WPF (limité à Windows), Avalonia UI est une solution multi-plateforme native pour le bureau.

Justification du choix par rapport aux exigences :

- Réponse à l'exigence [2.3 \(Multi-plateforme\)](#) : Avalonia permet de compiler le même code source pour qu'il s'exécute nativement et avec le même rendu visuel sur Windows, macOS et les distributions Linux.
- Indépendance du système : Il dessine sa propre interface (rendu Skia), ce qui garantit que l'application aura exactement la même apparence (pixel-perfect) quel que soit l'OS d'exécution.
- Flexibilité (XAML) : L'utilisation du langage de description XAML permet de séparer proprement le design de la logique de code.

2.6 Architecture logicielle

Cette application étant une application mono-utilisateur, il n'est donc pas nécessaire d'avoir une scalabilité ou de gestion de microservice. La base de données étant gérée directement par l'application (SQLite), une architecture monolithique est le choix le plus facile et logique.

L'architecture retenue pour ce projet est de type Monolithique.

Compte tenu de la nature du projet, une architecture distribuée (type microservices ou client-serveur web) ajouterait une complexité technique injustifiée sans apporter de valeur ajoutée aux besoins de l'étudiant.

Le choix du monolithe s'impose pour les raisons suivantes :

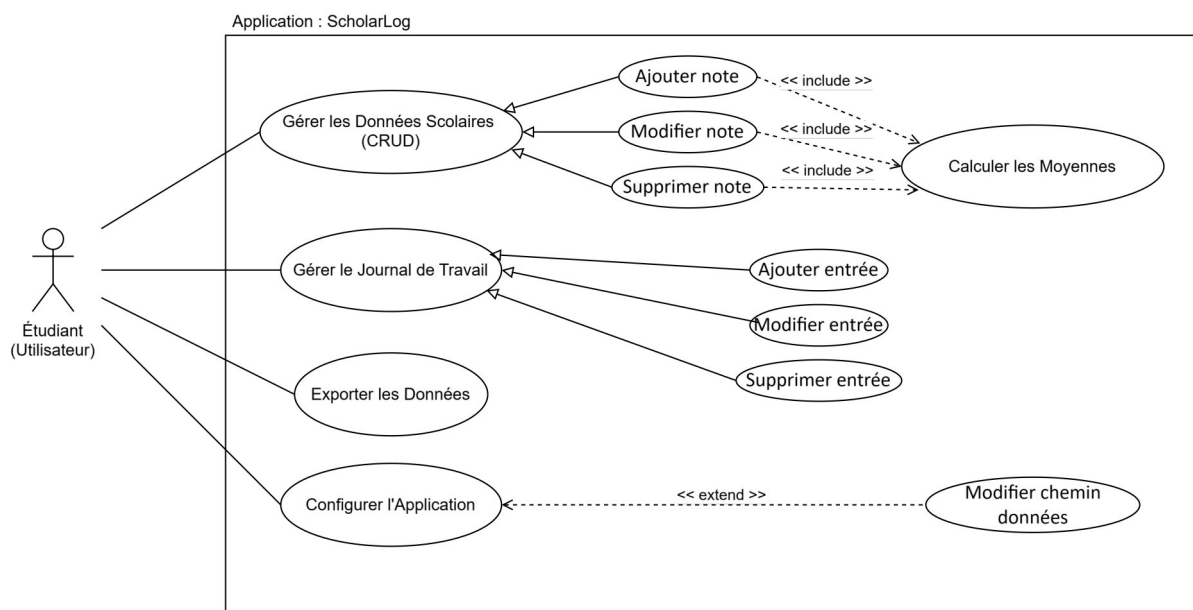
- Simplicité de développement et d'utilisation: L'application est compilée en un seul exécutable (ou un ensemble de fichiers liés). L'installation se résume à la copie des fichiers.
- Performance (Latence nulle) : La base de données SQLite étant embarquée (In-Process), il n'y a aucun appel réseau. Les opérations de lecture et d'écriture se font directement sur le système de fichiers, garantissant une fluidité maximale.
- Indépendance : L'application est totalement autonome. Elle ne dépend d'aucun service tiers ni d'aucune connexion internet pour fonctionner.

2.6.1 Architecture interne

l'organisation interne du code source suit le patron de conception MVVM (Modèle-Vue-ViewModèle). Ce choix est imposé par l'utilisation du framework Avalonia UI et permet une séparation stricte entre l'interface graphique et la logique métier, facilitant ainsi la maintenance et l'évolution du logiciel.

2.7 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation ci-dessous modélise le comportement de l'application du point de vue de l'utilisateur. Il permet d'identifier les grandes fonctionnalités attendues ainsi que les interactions entre l'acteur principal et le système.



L'application étant conçue pour un usage local et individuel, l'étudiant (utilisateur) est le seul acteur.

Gestion des Données Scolaires (CRUD) Ce cas d'utilisation central regroupe toutes les opérations liées aux modules et aux cours.

Les actions « Ajouter note », « Modifier note » et « Supprimer note » sont des spécialisations de la gestion globale. La fonctionnalité « Calculer les Moyennes » est incluse systématiquement. Cela signifie que dès qu'une note est ajoutée, modifiée ou supprimée, le système doit obligatoirement recalculer les moyennes pondérées pour garantir la cohérence des résultats affichés.

Gestion du Journal de Travail Similaire à la gestion des notes, ce cas permet le suivi des heures.

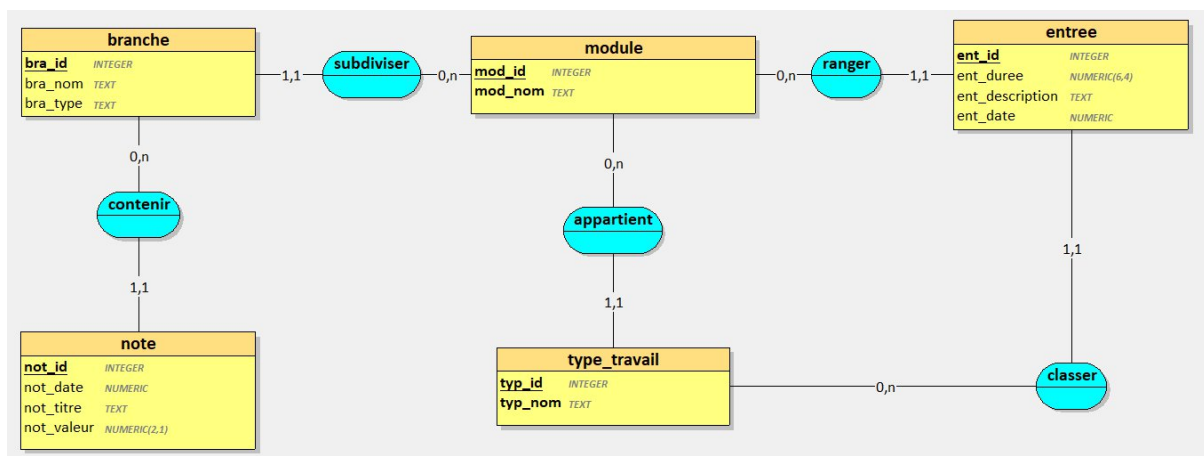
Les actions d'ajout, de modification et de suppression d'entrées journalisées sont représentées comme des cas spécifiques héritant de la gestion du journal.

Configuration de l'application : Le cas « Modifier chemin données » est une extension du cas « Configurer l'Application ». Cette relation indique que la modification de l'emplacement de sauvegarde est une action optionnelle qui ne s'exécute que sous certaines conditions (volonté spécifique de l'utilisateur de déplacer la base SQLite).

Exportation : permet à l'utilisateur d'extraire ses informations (au format CSV ou tableau) pour une utilisation externe.

2.8 Modèle Conceptuel de Données (MCD)

Le Modèle Conceptuel de Données (MCD) ci-dessous formalise la structure des informations gérées et stockées dans la base de donnée local. Il définit les entités, leurs attributs et les relations qui les unissent. Cette modélisation imposera des choix dans la conception des classes.



Voici le dictionnaire de données :

Entités	Champ	Type de donnée	Exemple
module	PK : mod_id	Entier non-signé	1, 2, 3 ...
	mod_nom	Texte	"M0 – Connaissances de base"
branche	PK : bra_id	Entier non-signé	1, 2, 3 ...
	bra_nom	Texte	"Mathématique"
	bra_type	Texte / Enum	"M", "TM", "PM"
	FK : mod_id	Entier non-signé	1, 2, 3 ...
type_travail	PK : typ_id	Entier non-signé	1, 2, 3 ...
	typ_nom	Texte	"Documentation"
	FK : mod_id	Entier non-signé	1, 2, 3 ...
	FK : typ_id	Entier non-signé	1, 2, 3 ...
entree	PK : ent_id	Entier non-signé	1, 2, 3 ...
	ent_duree	Nombre à virgule	1.5, 0.5, 1.6 ...
	ent_description	Texte	"Étude du cahier des charges"
	ent_date	Date	2026-03-10 (YYYY-MM-DD)
	FK : mod_id	Entier non-signé	1, 2, 3 ...
note	PK : not_id	Entier non-signé	1, 2, 3 ...
	not_date	Date	2026-03-10 (YYYY-MM-DD)
	not_titre	Texte	"TE02 – PET grammar"
	not_valeur	Nombre à virgule	4.5, 5.0, 5.5 (entre 1 et 6)
	FK : bra_id	Entier non-signé	1, 2, 3 ...

PK = Primary Key (Clé primaire), FK = Foreign Key (Clé étrangère).

2.9 Interface utilisateur

L'interface de l'application ScholarLog a été conçue pour être ergonomique. Elle doit permettre à l'étudiant d'accéder aux fonctions principales, comme la saisie d'une note, en moins de 3 clics. La navigation s'effectue via un menu latéral offrant un accès direct aux différentes pages de l'application.

2.9.1 Choix de design

La maquette de l'interface a été initialement conçue sur l'outil Penpot avec une volonté forte de proposer un design minimaliste et épuré. L'objectif principal est de ne pas surcharger l'étudiant d'informations inutiles et de rendre l'utilisation quotidienne la plus intuitive possible.

- Thème sombre (Dark Mode) : L'application utilise un thème sombre, offrant une interface moderne et réduisant la fatigue visuelle. Si le temps le permet, un mode clair serait envisageable (réalisé dans la suite du document [3.4.1](#)).
- Affichage par cartes (Cards) : Dans la vue principale, les informations sont structurées sous forme de blocs ou de cartes pour chaque module (ex: M0, M1, M2). Ce choix permet une lecture aérée et rapide des données.
- Code couleur et indicateurs visuels : Pour faciliter la lecture des résultats, les notes sont mises en évidence par un code couleur (les modules en réussite apparaissent en vert, tandis que ceux en difficulté ressortent en rouge). Des flèches de tendance accompagnent également les moyennes pour indiquer instantanément si les résultats sont en hausse, en baisse ou stables.

2.9.2 Pages

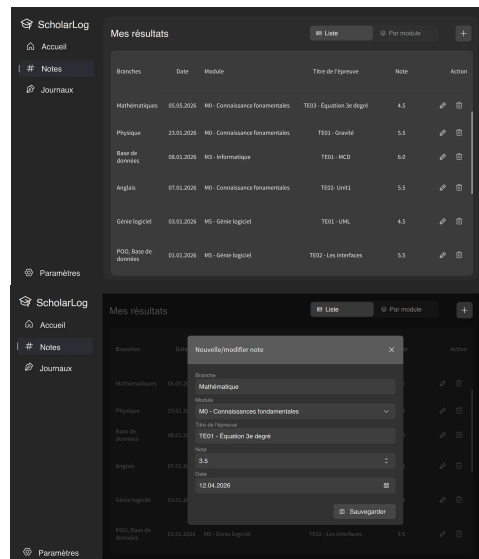
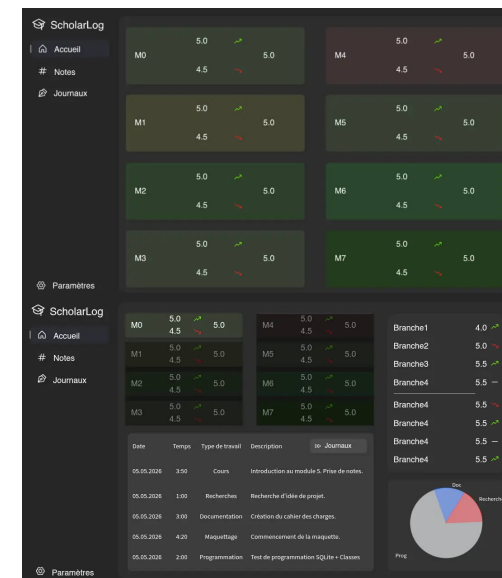
Ce chapitre détaille les différentes vues (pages) qui composent l'interface utilisateur de l'application.

(voir page suivantes)

2.9.2.1 Accueil

La page d'accueil agit comme un tableau de bord interactif. Dès l'ouverture de l'application, l'utilisateur a une vue d'ensemble immédiate sur ses performances globales.

- Vue globale : Un panneau central affiche les moyennes générales pour chaque module (M0, M1, etc.) sous forme de cartes synthétiques, accompagnées d'indicateurs de tendance (flèches vertes pour une hausse, rouges pour une baisse).
- Vue détaillée : Au clic sur un module spécifique, l'interface s'adapte dynamiquement pour afficher les détails associés sur la partie droite et inférieure de l'écran :
 - Les moyennes individuelles de chaque branche composant ce module.
 - Le journal de travail récent lié au module (comprenant la date, le temps passé, le type de travail et une description).
 - Un graphique circulaire illustrant visuellement la répartition du temps de travail par catégorie (ex: Documentation, Recherche, Programmation).



2.9.2.2 Note

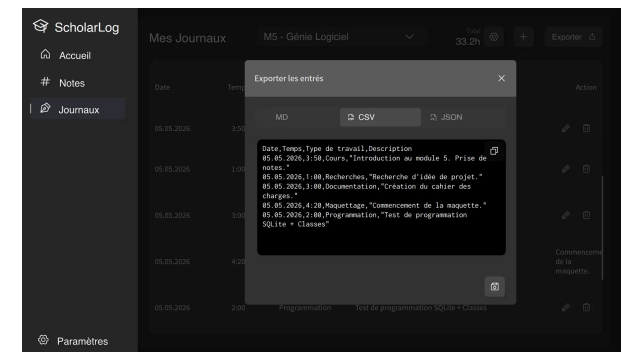
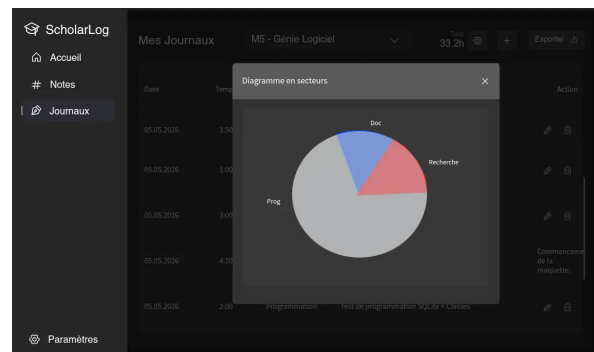
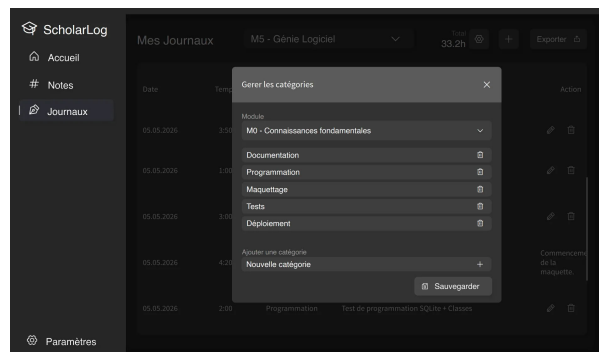
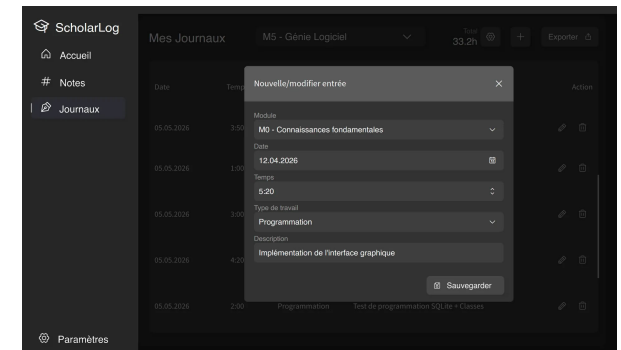
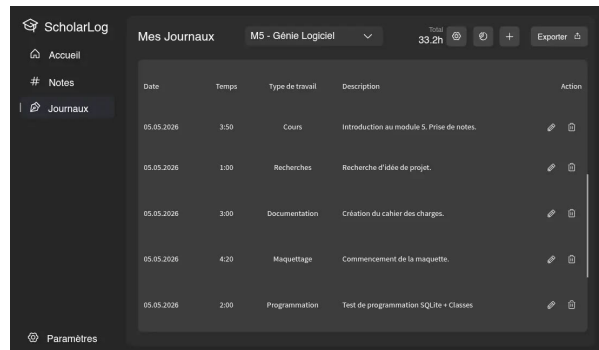
La page "Notes" est l'espace central dédié à la gestion des résultats scolaires. Elle permet à l'utilisateur d'assurer le suivi complet de ses évaluations.

- Gestion des évaluations : L'interface permet la saisie d'une nouvelle note (via un bouton d'ajout rapide "+"), ainsi que la modification ou la suppression de résultats existants grâce à des icônes d'action dédiées en bout de ligne.
- Affichage structuré : Les résultats sont présentés sous forme de liste claire reprenant la branche, la date, le module, le nom de l'épreuve et la note obtenue.
- Filtres de vue : Des onglets permettent de basculer facilement entre une vue globale en « Liste » et une vue triée « Par module » pour faciliter la lecture et la recherche.

2.9.2.3 Journaux

La page “Journaux” est conçue pour le suivi rigoureux du temps de travail de l'étudiant tout au long de ses modules.

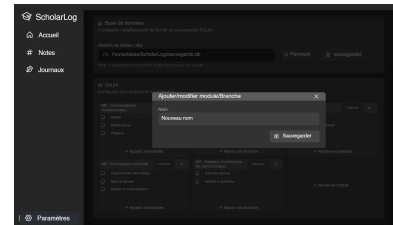
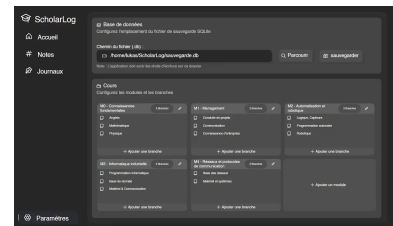
- Suivi des activités : Elle permet l'ajout, la modification et la suppression d'entrées journalisées pour documenter le temps passé sur chaque tâche.
- Catégorisation : L'utilisateur a la possibilité d'éditer les catégories d'entrées (types de travail) afin d'organiser son suivi selon ses propres méthodes d'apprentissage.
- Exportation : C'est également depuis ce menu que l'utilisateur peut déclencher l'exportation de ses données pour une sauvegarde externe ou une analyse sur un tableur.



2.9.2.4 Paramètres

La page “Paramètres” regroupe la configuration technique de l'application ainsi que la gestion structurale de la formation. La page est divisée en deux sections distinctes :

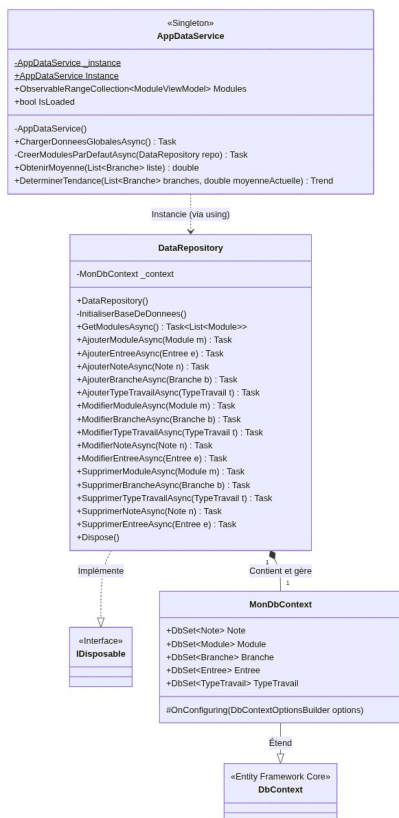
- Base de données : L'utilisateur peut configurer le chemin d'accès exact du fichier de sauvegarde SQLite local. Des boutons permettent de parcourir l'ordinateur pour sélectionner le dossier cible et de sauvegarder ce choix, assurant à l'élève un contrôle total sur ses données.
- Cours (Modules et Branches) : Cette section permet de structurer l'environnement d'apprentissage. Sous forme de cartes modifiables, l'utilisateur peut assurer l'ajout, l'édition et l'organisation complète des modules de sa formation et des branches qui y sont associées.



3. Conception de l'application

Cette phase consiste à traduire les besoins et les données précédemment définis en une solution technique concrète.

3.1 Architecture globale et Modélisation



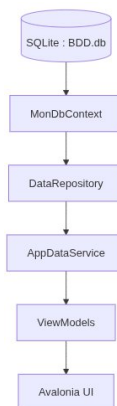
Les diagrammes de classes ci-dessous représentent la structure des classes de l'application.

Il détaille l'architecture orientée objet, définissant les attributs, les méthodes et les relations entre les différents composants logiciels.

Ce modèle sert de plan direct pour le développement (codage) des entités.

Contrairement à un modèle de données classique, les entités de base (Module, Branche, Note, Entree, TypeTravail) héritent toutes de ObservableObject.

Cela permet de notifier automatiquement l'interface utilisateur de tout changement. De plus, des Modèles de Vue (ViewModels) étendent ces entités pour l'affichage, en y ajoutant des propriétés dynamiques (par exemple, le calcul des moyennes comme AvgTheory ou les tendances Trend).



3.2 Accès aux données

L'accès et la manipulation des données persistantes reposent sur Entity Framework Core (EF Core) couplé à une base SQLite. L'architecture d'accès aux données est divisée en trois composants principaux qui se superposent :

3.2.1 Le Contexte de Base de Données (MonDbContext)

La classe MonDbContext assure la liaison entre les objets C# (DbSet) et les tables SQLite.

- Optimisation des performances : Le DbContext utilise un modèle compilé (CompiledModels) pour des performances accrues au lancement de l'application.
- Configuration : Elle configure la création du fichier BDD.db dans le répertoire de l'exécutable.

Attention - Maintenance : Ce projet n'utilise pas les EF Migrations classiques. Si la structure des entités est modifiée, il faut impérativement régénérer les modèles compilés via la commande :

```
dotnet ef dbcontext optimize -c MonDbContext -o CompiledModels --namespace ScholarLog.Data.CompiledModels.
```

3.2.2 Le dépôt de Données (DataRepository)

Afin d'isoler la logique d'accès aux données, l'application implémente le pattern Repository.

- Opérations CRUD : Il centralise les opérations Create, Read, Update, Delete de manière asynchrone.
- Chargement optimisé (Eager Loading) : La méthode GetModulesAsync utilise Include, ThenInclude et l'optimisation AsSplitQuery pour récupérer toute l'arborescence d'un module en une seule requête ciblée.

3.2.3 Le Service Global (AppDataService)

C'est le chef d'orchestre de la logique métier, implémenté sous forme de Singleton (AppDataService.Instance).

- Source de vérité (State Management) : Il charge et maintient la liste des modules en mémoire via une ObservableRangeCollection partagée par toute l'application.
- Logique métier : Il contient les algorithmes de calculs, comme les moyennes arithmétiques arrondies au 0.5 le plus proche et la détermination des tendances (Up, Down, Stable).

3.3 Le Modèle de Données

Le fichier models.cs constitue le cœur structurel de l'application. Il définit non seulement les entités du domaine métier (mappées à la base de données SQLite via Entity Framework Core), mais également les extensions nécessaires à l'interface utilisateur sous forme de ViewModels.

Pour faciliter l'interaction avec Avalonia UI, l'ensemble de ces classes hérite de ObservableObject, permettant ainsi la notification automatique des changements de propriétés à l'interface.

3.3.1 Les énumérations

Le modèle s'appuie sur des énumérations pour standardiser certaines données :

- TypeCours : Définit la nature d'une branche (M, TM, PM).
- Trend : Indique la tendance visuelle d'une moyenne pour le tableau de bord (Up, Down, Stable).

3.3.2 Les entités du domaine (Entity Framework Core)

Ces classes utilisent les annotations de données ([Table], [Key], [Column], [ForeignKey]) pour configurer la base de données relationnelle locale.

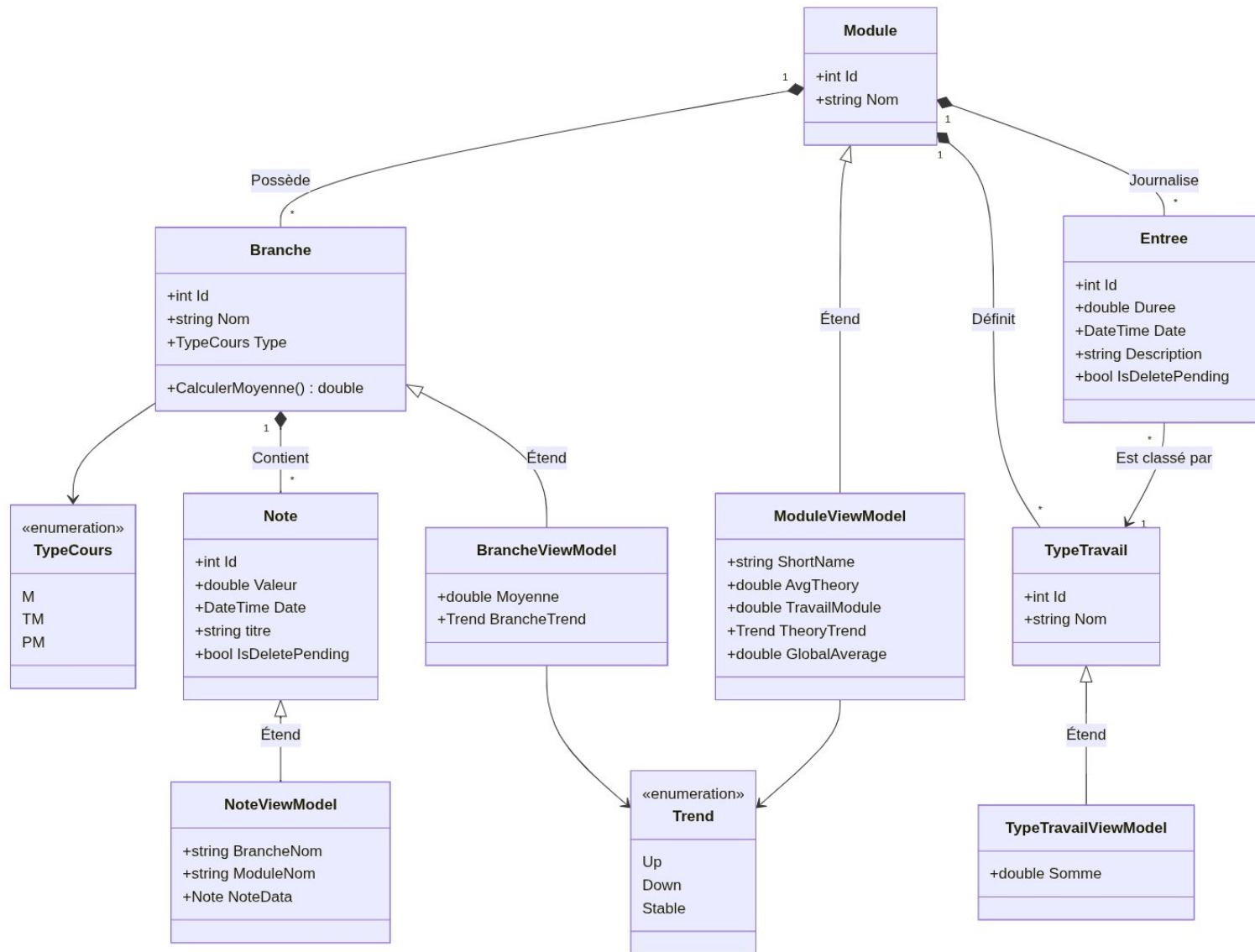
Entité	Rôle et caractéristiques
Module	Entité racine regroupant de manière hiérarchique ses branches, ses types de travaux personnalisés et son journal de travail.
Branche	Représente une subdivision d'un module. Elle intègre la méthode métier CalculerMoyenne() qui extrait la moyenne exacte des notes et l'arrondit automatiquement au demi-point (0.5) le plus proche.
Note	Stocke une évaluation (valeur entre 1 et 6, date, titre). Elle possède une propriété locale non mappée en base (IsDeletePending) pour gérer l'état d'affichage dans l'interface.
Entree	Documente une tâche du journal de travail, incluant la durée, la date, la description et les liaisons vers le module et le type de travail concernés.
TypeTravail	Catégorie de travail associée au journal (ex: Programmation, Recherche). La liste est spécifique à chaque module pour éviter l'affichage de données non pertinentes.

3.3.3 Les Modèles de Vue (ViewModels)

Afin de respecter le pattern MVVM sans surcharger la base de données, l'application utilise des classes étendues pour enrichir les entités de base avec des propriétés dynamiques calculées spécifiquement pour l'affichage.

ViewModel	Propriétés ajoutées pour l'interface
ModuleViewModel	Centralise les calculs du tableau de bord : abréviation du nom (ShortName), moyennes théoriques et globales (AvgTheory, GlobalAverage), tendance (TheoryTrend) et temps de travail total (TravailModule).
BrancheViewModel	Ajoute la note moyenne formatée (Moyenne) et l'indicateur visuel de progression (BrancheTrend).
TypeTravailViewModel	Intègre la propriété Somme pour cumuler et afficher le total des heures travaillées par catégorie.
NoteViewModel	Embarque les métadonnées de contexte (noms de la branche et du module parents) pour faciliter un affichage compréhensible dans les listes globales de l'application.

3.3.4 Diagramme des entités (Model)



3.4 Implémentation de la page Paramètres

Cette section détaille la réalisation technique de l'interface de configuration (SettingsPage.axaml et son code-behind), développée avec le framework Avalonia UI. Cette vue permet à l'utilisateur de configurer les éléments globaux du programme.

3.4.1 Gestion de l'apparence (Thème)

Bien que l'interface soit synchronisé avec le thème de l'appareil, l'interface intègre un commutateur (ToggleSwitch) permettant à l'utilisateur de forcer le basculement entre le mode clair et le mode sombre. Ce choix modifie le thème global de l'application.

Pour garantir une parfaite intégration visuelle, l'ensemble des éléments (couleurs de fond, bordures, typographies) utilise des ressources dynamiques (DynamicResource) qui proviennent du dictionnaire principal de l'application.

3.4.2 Gestion de la base de données (Chemin d'accès)

Une carte dédiée permet de configurer l'emplacement du fichier de sauvegarde SQLite (.db).

Sélection de fichier : Un bouton "parcourir" est lié à une commande (BrowseFileCommand) qui ouvre un sélecteur de fichiers natif à l'OS utilisé.

Validation des saisies : Le système vérifie la validité du chemin fourni par l'utilisateur.

Gestion des erreurs : Si le fichier est introuvable, une bulle d'alerte contextuelle (Flyout) s'affiche directement sous le champ de texte. Cette alerte informe visuellement l'utilisateur que le fichier est introuvable et précise que la modification du chemin n'a pas été enregistrée.

3.5 Design et ressources globales

Afin de garantir une interface utilisateur harmonieuse et maintenable, l'ensemble des styles globaux, des ressources visuelles et des thèmes de l'application est centralisé au sein du fichier App.axaml. Cette approche centralisée permet de définir une seule fois les polices, les dimensions, les couleurs et les icônes, puis de les réutiliser dans toutes les vues du projet.

3.5.1 Gestion des Thèmes (Mode Clair / Mode Sombre)

L'application intègre nativement une gestion du thème clair et du thème sombre. Les couleurs sont attribuées dynamiquement grâce à l'utilisation de dictionnaires de thèmes (ResourceDictionary.ThemeDictionaries).

Chaque thème redéfinit un ensemble de variables clés, telles que les couleurs d'arrière-plan (PrimaryBackground, CardBackground), les couleurs de texte (PrimaryForeground, SecondaryForeground) ou encore les couleurs de bordure (CardBorder). Des ajustements spécifiques sont également prévus selon le système d'exploitation, avec par exemple un arrière-plan dédié pour Linux (PrimaryBackgroundLinux). Enfin, l'icône de la fenêtre de l'application (AppIcon) change dynamiquement pour s'adapter au thème actif.

3.5.2 Typographie et Système d'Icônes

Les polices sont intégrées directement dans l'application et chargées depuis le répertoire local Assets/Fonts.

L'interface utilise la police Helvetica pour l'affichage de ses textes. Pour l'iconographie, le projet s'appuie sur la police Phosphor.

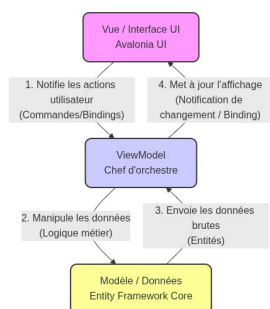
Les icônes affichées (comme la maison, l'engrenage, le disque dur, ou le soleil et la lune) ne sont pas des images classiques, mais des caractères Unicode spécifiques interprétés par cette police. Cette méthode technique garantit un rendu vectoriel net à n'importe quelle échelle et simplifie la modification des couleurs lors des changements de thème.

3.5.3 Cohérence Visuelle et Dimensions

Pour éviter les valeurs "en dur" dispersées dans le code des différentes pages, des constantes structurelles ont été définies globalement. Des variables standardisées (Small, Normal, Medium, Large, Logo) régissent l'échelle typographique et la taille des éléments visuels de manière uniforme.

La courbure des éléments de l'interface est également uniformisée grâce aux ressources CardCornerRadius et FieldCornerRadius, assurant un design cohérent entre les conteneurs principaux et les champs de saisie. De plus, les couleurs d'état, comme ErrorForeground (définie en rouge pour signaler une erreur), sont stockées dans ce même dictionnaire pour garantir un retour visuel constant à l'utilisateur.

3.6 Architecture MVVM et Logique de Présentation



Pour garantir un code propre, testable et maintenable, l'application s'appuie sur le pattern d'architecture MVVM (Model-View-ViewModel). Le principe central est de séparer strictement l'interface graphique (la Vue) de la logique métier et des données (le Modèle).

Cela permet d'avoir un code propre, testable, et où l'interface graphique n'est pas mélangée avec la logique de la base de données.

Le ViewModel agit comme un chef d'orchestre intermédiaire entre ces deux couches :

- Il prépare et formate les données du Modèle pour qu'elles soient affichables par la Vue.
- Il intercepte les actions de l'utilisateur (comme les clics ou les saisies) par le biais de Commandes (par exemple, [RelayCommand]).
- Il notifie automatiquement la Vue lorsque les données changent, en utilisant notamment [ObservableProperty] fourni par le CommunityToolkit.Mvvm.

Voici la structure des ViewModels implémentés dans l'application :

- **ViewModelBase** : Cette classe hérite de `ObservableObject` et sert de parent à tous les autres ViewModels pour leur transmettre les capacités de notification de changement d'état.
- **MainWindowViewModel** : Il s'agit du chef d'orchestre principal. Il gère la navigation globale de l'application (basculer entre les différentes pages), contrôle l'ouverture du menu latéral, et orchestre la séquence de chargement initial.
- **HomeViewModel** : Il gère le tableau de bord (Accueil). Il agrège les données globales pour afficher des résumés, calcule les moyennes par branche, prépare les données pour le graphique circulaire (Donut), détermine les tendances, et permet la navigation via un système de messagerie (`WeakReferenceMessenger`).
- **NotesViewModel** : C'est le gestionnaire d'évaluations. Il s'occupe de la logique CRUD pour les notes, lie dynamiquement chaque note à sa branche ou son module parent, et synchronise ces opérations en direct avec la base de données SQLite.
- **JournalViewModel** : Il constitue le moteur du journal de travail étudiant. Il gère toutes les opérations d'ajout, de modification et de suppression (CRUD) des entrées de journal et de leurs catégories. Il embarque également la logique complexe de génération des rapports d'exportation (formats Markdown, CSV, JSON).
- **SettingsViewModel** : Il agit comme le gestionnaire des paramètres. Il contrôle la validation et la sauvegarde du chemin de la base de données locale, ainsi que la bascule dynamique entre le thème clair et sombre de l'interface.

3.7 Services et Gestion d'État Globale

Pour centraliser la logique métier transversale et maintenir un état cohérent durant toute l'exécution, l'application utilise un service global nommé AppDataService.

- Singleton : Ce service est implémenté sous forme de Singleton (AppDataService.Instance), agissant comme l'unique source de vérité (State Management) pour toute l'application.
- Gestion de la mémoire : Il charge et maintient la liste des modules en mémoire via une ObservableCollection qui est partagée par toute l'application.
- Initialisation : Il est responsable de l'initialisation asynchrone depuis la base de données, ou de la création de l'arborescence par défaut (modules de M0 à DIPL.) si la base est vide.
- Logique Métier : Il encapsule la logique métier complexe, telle que le calcul des moyennes arithmétiques (arrondies au 0.5 le plus proche) et la détermination des tendances de notes (en comparant la dernière note à la moyenne globale avec une marge de 0.2).

3.8 Optimisation et Déploiement de la Base de Données

Afin de garantir d'excellentes performances, notamment lors du lancement de l'application, l'accès aux données a été optimisé.

- Modèles Compilés : Le DbContext utilise un modèle compilé généré par Entity Framework Core pour accélérer le temps de démarrage. Cela est défini via l'instruction `.UseModel(ScholarLog.Data.CompiledModels.MonDbContextModel.Instance)`.
- Maintenance : Étant donné que le projet n'utilise pas les migrations classiques d'EF (la base est générée dynamiquement au premier lancement), une attention particulière est requise lors du développement. Si la structure des entités est modifiée (ajout de table, modification de colonne), il est impératif de régénérer les modèles compilés via la commande CLI .NET ci-dessous :

```
dotnet ef dbcontext optimize -c MonDbContext -o CompiledModels --namespace ScholarLog.Data.CompiledModels
```

3.9 Traitement et Gestion des Fichiers Jointes (PDF/CBZ)

Afin de ne pas alourdir la base de données SQLite tout en permettant à l'étudiant d'associer des évaluations numérisées à ses notes, une approche d'optimisation agressive a été mise en place.

Plutôt que de stocker des PDF bruts qui feraient exploser la taille de la base de données, l'application utilise la bibliothèque SkiaSharp pour traiter les documents. Lors de l'ajout d'un PDF, le fichier est lu, ses pages sont converties en images (redimensionnées à une définition maximale raisonnable, ex: 2.5Mp) puis compressées au format WebP (qualité 40-45). Ces images compressées sont ensuite regroupées dans une archive CBZ (Zip) stockée sous forme de BLOB (byte[]) dans la table NoteArchive. Cette technique permet de maintenir une empreinte de base de données extrêmement basse (estimation max ~60 Mo).

Pour garantir que l'interface (GUI) reste 100% fluide lors de la consultation d'un document, la décompression et l'affichage des pages sont gérés de manière asynchrone (Stream/In-Memory) via des tâches dédiées (Task.Run), libérant ainsi le thread principal (UiThread).

Cette optimisation a donné lieu à un PoC (Proof of Concept), ce document est nommé "LHO - M5 - PoC Scans.docx".

"En compressant intelligemment les documents en arrière-plan plutôt que de les sauvegarder de manière classique, l'application parvient à réduire l'espace de stockage nécessaire de plus de 80 %."

Objet	Original (PDF)	Restitution (PDF)	Stockage dans BDD (BLOB)	Gain dans la DB
Livre	95,5 Mo	50,0 Mo	17,28 Mo	82 %
Épreuve de 8 Pages	2,80 Mo	1,20 Mo	0,518 Mo	82 %
Épreuve de 7 Pages	2,00 Mo	0,20 Mo	0,319 Mo	84 %
Épreuve de 8 Pages	2,50 Mo	0,76 Mo	0,364 Mo	85 %

Voir document : "LHO - M5 - PoC Scans"

3.10 Exportation des données

Pour répondre à l'exigence fonctionnelle REQ-F-005, l'application intègre un module d'exportation accessible notamment depuis le journal de travail. Ce module (ModalExport) permet à l'utilisateur d'exporter ses données vers trois formats standardisés :

- MD (Markdown) : Pour une lecture propre et structurée, incluant la répartition des heures.
- CSV : Pour une intégration facile dans des tableurs externes (Excel, Calc).
- JSON : Pour garantir l'interopérabilité des données avec d'autres systèmes informatiques. La logique de génération est encapsulée dans le JournalViewModel, qui calcule les totaux et prépare le texte dynamiquement avant de le transmettre à la vue.

Point négatif : les graphiques de répartition des heures (donuts) ne sont pas exportés (voir points d'améliorations dans [7.3 Perspective d'évolution](#)).

3.11 Composants Utilitaires et d'Interface Graphique

Pour soutenir l'architecture MVVM et garantir les performances de l'interface graphique (Avalonia UI), l'application s'appuie sur plusieurs classes utilitaires développées spécifiquement pour le projet.

3.11.1 ViewLocator

Il s'agit d'un composant fondamental pour le routage dans Avalonia. Cette classe implémente l'interface `IDataTemplate` et agit comme un pont dynamique.

Lorsqu'un `ViewModel` (ex: `HomeViewModel`) est appelé par le chef d'orchestre, le `ViewLocator` résout dynamiquement le nom de la classe, instancie la vue correspondante (ex: `HomeView`) et l'affiche.

Cela permet de naviguer dans l'application sans jamais coupler fortement les Vues et les `ViewModels`.

3.11.2 WhiteSlideUpTransition

Pour répondre à l'exigence d'une interface ergonomique et moderne, cette classe personnalisée (implémentant `IPageTransition`) gère les animations lors des changements de page.

Elle orchestre de manière asynchrone un fondu d'opacité combiné à un glissement vertical (`TranslateTransform.YProperty`), rendant la navigation beaucoup plus fluide pour l'utilisateur.

3.11.3 ObservableRangeCollection

L'ajout d'éléments un par un dans une `ObservableCollection` classique déclenche un rafraîchissement visuel à chaque itération, ce qui peut geler l'interface lors du chargement initial de la base de données.

Cette collection personnalisée intègre une méthode `ReplaceAll` qui ajoute les données silencieusement, puis envoie une unique notification globale de réinitialisation (`NotifyCollectionChangedAction.Reset`) à l'interface, optimisant considérablement les temps de chargement.

3.11.4 Modules de Traitement des Fichiers (ScanHandlers)

Bien que la logique conceptuelle soit détaillée dans le document PoC Scans, le traitement technique réel repose sur plusieurs classes statiques dédiées, isolant ainsi la manipulation des flux et des threads de travail (Task.Run) :

- ImageProcessor :
Gère la manipulation brute des images via SkiaSharp. Elle contient la logique de redimensionnement asynchrone (ResizeToMaxPixelsAsync) basée sur le calcul de surface (Mégapixels), tout en gérant rigoureusement la destruction en RAM (Dispose()) des bitmaps originaux pour éviter les fuites de mémoire.
- PdfManager :
Encapsule la lecture et l'écriture des fichiers PDF. Elle génère de nouveaux PDF en peignant les images compressées sur des canvas virtuels, ou extrait les pages d'un PDF existant sous forme de flux asynchrone (IAsyncEnumerable).
- ArchiveManager :
Le cœur du moteur de compression. Cette classe génère des archives Zip/CBZ en mémoire (CreateCbzInMemoryAsync) en encodant les bitmaps en WebP à la volée. Elle s'occupe également du décodage inverse, permettant à l'application de lire des données ultra-compressées sans solliciter le disque dur.
- DirectoryManager :
Facilite les imports et exports massifs. Elle lit de manière non-bloquante les images contenues dans un dossier système et permet leur sauvegarde physique en déléguant l'encodage à des threads secondaires pour protéger le fil d'exécution de l'interface utilisateur.

3.12 Composants Visuels Personnalisés (UserControls)

Afin d'enrichir l'interface utilisateur tout en maintenant une architecture modulaire, le projet intègre plusieurs contrôles personnalisés (UserControls) développés spécifiquement avec Avalonia UI. Ces composants encapsulent leur propre logique d'affichage et d'interaction.

3.12.1 DonutChartControl (Graphique de répartition)

Ce composant est responsable de l'affichage du graphique en anneau (utilisé pour illustrer la répartition du temps de travail dans les journaux).

3.12.1.1 Raison d'être de l'implémentation native

Lors du développement du module précédent (M3), l'utilisation de bibliothèques graphiques tierces (telles que LiveCharts2) a mis en évidence des problèmes de conflits de dépendances inter-plateformes (notamment sous Linux) et une augmentation substantielle du poids de l'exécutable final.

Pour respecter l'empreinte mémoire stricte du projet et garantir son indépendance, ce graphique en anneau (Donut Chart) a été généré de toutes pièces. Il s'appuie exclusivement sur la géométrie analytique et le dessin vectoriel natif d'Avalonia (StreamGeometry).

3.12.1.2 Modélisation mathématique du tracé

L'anneau est modélisé informatiquement comme une partition d'une couronne circulaire (comprise entre un rayon intérieur r et extérieur R). Chaque secteur correspond à une proportion p_i du total, traduite en amplitude angulaire : $\Delta\theta_i = p_i * 2\pi$.

Repère d'étude et conventions angulaires :

Le rendu s'effectue dans le repère cartésien bidimensionnel de l'écran où l'origine $O(0,0)$ est située au coin supérieur gauche de la zone de dessin (Canvas). Une spécificité fondamentale de l'infographie est que l'axe des ordonnées (Y) est orienté vers le bas. Ainsi :

- Un angle positif décrit un mouvement dans le sens horaire (inversion du sens trigonométrique standard).
- Pour des raisons ergonomiques et de convention de lecture, le tracé commence à la position "12 heures". L'angle initial θ_0 est donc défini par :

$$\theta_0 = -\frac{\pi}{2}$$

Conversion Polaires vers Cartésiennes

Pour dessiner chaque secteur, l'algorithme calcule les coordonnées paramétriques de ses 4 sommets (départ/arrivée sur l'arc extérieur, puis intérieur) par rapport au centre du canevas $C(X_c, Y_c)$:

$$X = X_c + Rayon * \cos(\theta) \qquad Y = Y_c + Rayon * \sin(\theta)$$

Synthèse vectorielle (PathData)

Le polygone curviligne est ensuite tracé et fermé via une séquence géométrique SVG :

- M (Move) : Déplacement au point de départ sur le rayon externe.
- A (Arc) : Tracé de l'arc extérieur. La logique calcule dynamiquement le *Large Arc Flag* (1 si $\Delta\theta > \pi$, sinon 0) pour garantir la justesse de la concavité.
- L (Line) : Jonction radiale vers le rayon interne r .
- A (Arc) : Tracé de l'arc intérieur, avec un balayage inversé (*Sweep Flag* = 0) pour remonter vers l'angle de départ.
- Z (Close) : Fermeture automatique de la surface pour permettre l'injection de la couleur.

3.12.2 ModalExport (Interface d'exportation de données)

Ce composant agit comme une fenêtre modale superposée à l'interface principale pour gérer l'exportation des données (Journaux de travail).

- Sélecteur de format :
Il gère dynamiquement les changements d'état entre les différents formats d'exportation (MD, CSV, JSON) via des RadioButton.
- Intégration OS Native :
Ce code-behind prend en charge des actions nécessitant une interaction directe avec le système d'exploitation, ce qui est volontairement découplé des *ViewModels*. Il utilise l'API Clipboard d'Avalonia pour copier les données en un clic, et le StorageProvider pour ouvrir une boîte de dialogue native (SaveFileDialogAsync) permettant à l'utilisateur d'enregistrer le fichier sur son disque dur.

3.12.3 PdfViewerControl (Visionneuse PDF et Scans optimisée)

C'est le composant central du Proof of Concept (PoC) de traitement des scans. Il est conçu pour afficher les documents sans geler l'interface ni saturer la mémoire vive (RAM).

- Gestion mémoire ultra-légère :
Le contrôle reçoit un BLOB compressé depuis la base de données. Il stocke les pages sous forme d'archives compressées WebP en mémoire (List<byte[]>) et ne décode qu'une seule page image à la fois (_imageCourante) lors du visionnage.
- Navigation avancée :
Il prend en charge la navigation par boutons ainsi que par des raccourcis clavier (Flèches, Espace, Home, End).
- Manipulation de l'image (Pan & Zoom) :
L'utilisateur peut se déplacer dans l'image grâce à une fonctionnalité "cliquer-glisser" ("Drag"). Le zoom est particulièrement travaillé : il intercepte la molette de la souris en mode *Tunnel* (stratégie Avalonia RoutingStrategies.Tunnel) avec la touche "Control" pour empêcher le défilement natif et forcer un redimensionnement centré sur le curseur de la souris.
- Exportation inverse :
Il permet à l'utilisateur de cliquer sur un bouton pour reconstruire et télécharger un vrai fichier .pdf à partir des données compressées en RAM, en passant par le PdfManager.

3.12.4 LoadingView (Écran d'attente)

Un composant simple et réutilisable qui affiche un écran de chargement.

Il utilise une barre de progression (ProgressBar) configurée avec la propriété `IsIndeterminate="True"` pour signaler visuellement à l'utilisateur qu'un traitement asynchrone est en cours en arrière-plan sans bloquer l'application.

3.13 Configuration Globale et Fichiers de Ressources (App.xaml)

Afin d'assurer une cohérence visuelle parfaite dans toute l'application et de faciliter sa maintenance, l'ensemble des styles globaux, des variables de dimensionnement et des thèmes est centralisé dans le fichier de ressource principal App.xaml. Son *code-behind* (App.xaml.cs) se charge quant à lui d'orchestrer le cycle de vie de l'application.

3.13.1 Le Dictionnaire de Ressources (App.xaml)

Ce fichier fait office de "feuille de style" globale pour l'interface Avalonia UI. Il élimine le besoin d'écrire des valeurs en dur (*hardcoding*) dans les différentes vues de l'application en définissant un système de variables réutilisables (via `StaticResource` et `DynamicResource`).

- Polices et Typographie :
Le projet embarque ses propres polices (stockées dans le dossier Assets/Fonts) pour garantir un rendu identique sur tous les systèmes d'exploitation. L'application utilise Helvetica pour le texte standard et Phosphor pour l'iconographie. Les tailles de texte sont standardisées via des clés explicites (de `FontSizeSmall` à `FontSizeLarge`).
- Système d'Icônes Vectorielles :
Plutôt que d'utiliser de multiples images (PNG/SVG) lourdes à charger, les icônes de l'interface (ex: `IcôneHome`, `IcôneSave`, `IcôneMoon`) sont définies sous forme de caractères Unicode (, etc.). Liées à la police Phosphor, ces icônes se comportent comme du texte : elles sont vectorielles, redimensionnables sans perte de qualité et adoptent automatiquement la couleur définie par le thème actif. Code Unicode disponible sur <https://phosphoricons.com/>.
- Cohérence Géométrique :
Les rayons de courbure des éléments de l'interface sont uniformisés grâce à des variables globales de type `CornerRadius` (`CardCornerRadius`, `ControlCornerRadius`, `FieldCornerRadius`).
- Gestion des Thèmes (Clair / Sombre) :
Le fichier définit un `ThemeDictionaries` contenant deux dictionnaires distincts (Light et Dark). Chaque dictionnaire attribue des codes couleurs spécifiques (Background, Foreground, Borders) à des clés communes (`PrimaryBackground`, `PrimaryForeground`, etc.). Cela permet à l'interface de basculer instantanément d'un mode à l'autre en modifiant simplement la ressource pointée par `DynamicResource`. L'icône de la fenêtre (`AppIcon`) change également dynamiquement selon le thème.

4. Tests

Ce chapitre présente les procédures de validation mises en œuvre pour garantir la robustesse, la fiabilité et la conformité de l'application ScholarLog face aux exigences fonctionnelles et techniques définies initialement.

4.1 Configuration & Paramètres

ID Test	Catégorie	Fonctionnalité ciblée	Action à réaliser (Scénario)	Résultat attendu	Statut (OK/KO)	Remarques
CFG-01	Base de données	Déplacement BDD	Aller dans Paramètres > Base de données. Cliquer sur "Parcourir" et choisir un nouveau dossier valide.	Le chemin d'accès se met à jour, l'application recharge les données depuis ce nouvel emplacement.	OK	
CFG-02	Base de données	Gestion d'erreur BDD	Indiquer manuellement ou forcer une erreur pour simuler un fichier introuvable.	Une bulle d'alerte rouge (Flyout) "Le fichier est introuvable" s'affiche sous le chemin.	OK	Difficile à reproduire. Produit via code.
CFG-03	Apparence	Thème sombre/clair	Dans Paramètres > Apparence, basculer le <i>ToggleSwitch</i> du mode sombre/clair.	L'interface bascule instantanément, les couleurs et icônes (soleil/lune) s'adaptent.	OK	
CFG-04	Apparence	Mode performance	Désactiver les effets visuels dans Paramètres > Performances.	Fond transparent et orbes de décoration désactivés.	OK	

4.2 Gestion des Modules et Branches (CRUD)

ID Test	Catégorie	Fonctionnalité ciblée	Action à réaliser (Scénario)	Résultat attendu	Statut (OK/KO)	Remarques
MOD-01	Structure	Création Module	Dans Paramètres > Cours, cliquer sur "Ajouter un module".	Une nouvelle carte de module est créée.	OK	REQ-F-001
MOD-02	Structure	Édition Module	Cliquer sur l'icône "Crayon" d'un module. Modifier son nom et ajouter une branche.	Le modal s'ouvre. Le nom est mis à jour et la nouvelle branche apparaît dans la liste.	OK	REQ-F-001
MOD-03	Structure	Suppression Module	Cliquer sur l'icône "Corbeille" d'un module.	Un dialogue de confirmation ("Confirmation requise") s'affiche. Valider supprime le module.	OK	REQ-F-001

4.3 Exportation des données

ID Test	Catégorie	Fonctionnalité ciblée	Action à réaliser (Scénario)	Résultat attendu	Statut (OK/KO)	Remarques
EXP-01	Export	Ouverture Modale	Depuis le Journal ou les Notes, cliquer sur "Exporter".	Le modal s'ouvre, le format Markdown (MD) est sélectionné par défaut, et un aperçu s'affiche.	OK	REQ-F-005
EXP-02	Export	Changement de format	Basculer entre MD, CSV et JSON via les RadioButtons.	L'aperçu textuel se met à jour en direct selon le format choisi.	OK	REQ-F-005
EXP-03	Export	Contexte (Global/Module)	Dans la page Journal, vérifier la différence d'export (Module sélectionné vs Ctrl+Clic sur Exporter).	Le nom du fichier suggéré change (JournalDeTravail_ALL vs JournalDeTravail_MOD) et le contenu s'adapte.	OK	

4.4 Gestion des Notes

ID Test	Catégorie	Fonctionnalité ciblée	Action à réaliser (Scénario)	Résultat attendu	Statut (OK/KO)	Remarques
NOT-01	Notes (CRUD)	Ajout et limite de note	Ajouter une note. Saisir une valeur inférieure à 1 ou supérieure à 6.	Le champ NumericUpDown corrige automatiquement la valeur pour rester entre 1.0 et 6.0.	OK	REQ-F-002
NOT-02	Notes (CRUD)	Ajout avec date	Saisir une note avec une date spécifique ou laisser vide.	Si aucune date n'est sélectionnée, la date du jour est appliquée par défaut.	OK	REQ-F-002
NOT-03	Notes (CRUD)	Suppression sécurisée	Cliquer une seule fois sur l'icône "Corbeille" d'une note.	L'icône devient rouge et initie un délai de 3 secondes sans supprimer la note. Supprime la note si <3 secondes	OK	REQ-F-002
NOT-04	Notes (CRUD)	Annulation suppression	Attendre la fin des 3 secondes après le premier clic.	L'icône redevient normale, la note n'est pas supprimée.	OK	
NOT-05	Notes (CRUD)	Suppression rapide	Maintenir Ctrl, Maj ou Alt et cliquer sur la corbeille.	La note est supprimée instantanément sans délai.	OK	
NOT-06	Scans PDF	Ajout document PDF	Lors de l'édition d'une note, cliquer sur "Parcourir" pour joindre un PDF.	L'explorateur natif de l'OS s'ouvre, n'autorisant que les fichiers .pdf.	OK	PoC Scans
NOT-07	Scans PDF	Lecture document	Cliquer sur l'icône "Document" d'une note contenant un PDF.	Le PdfViewerControl s'ouvre en modal (Aperçu) et affiche le document sans figer l'UI.	OK	PoC Scans
NOT-08	Affichage	Filtres de vue	Basculer entre l'affichage "Liste" et "Par module" avec les RadioButtons.	L'affichage change entre la liste globale et le tri par branche/module.	OK	

4.5 Journal de Travail

ID Test	Catégorie	Fonctionnalité ciblée	Action à réaliser (Scénario)	Résultat attendu	Statut (OK/KO)	Remarques
JOU-01	Entrées	Ajout journal	Cliquer sur le "+" et remplir la durée, la date, le type et la description.	La nouvelle entrée s'affiche dans la liste du module sélectionné.	OK	REQ-F-004
JOU-02	Types	Édition catégories	Cliquer sur l'icône "Engrenage" puis ajouter et renommer un type de travail.	Le modal s'ouvre, les nouveaux types s'ajoutent et sont disponibles dans les listes déroulantes.	OK	REQ-F-004
JOU-03	Types	Suppression catégorie	Demander la suppression d'une catégorie (Type de travail).	Un message d'alerte rouge apparaît prévenant des conséquences.	OK	
JOU-04	Entrées	Suppression sécurisée	Comme pour les notes, tester le clic simple (3s) et le clic + Ctrl sur la corbeille.	Le comportement est identique : 3s de délai de sécurité ou suppression immédiate avec modificateur.	OK	REQ-F-004
JOU-05	Visualisation	Graphique de répartition	Cliquer sur l'icône de graphique ("Donut").	Le graphique en anneau s'ouvre en modal et reflète fidèlement la répartition des heures du module.	OK	

4.6 Tableau de bord & Moyennes (Accueil)

ID Test	Catégorie	Fonctionnalité ciblée	Action à réaliser (Scénario)	Résultat attendu	Statut (OK/KO)	Remarques
ACC-01	Calculs	Moyennes automatiques	Vérifier les moyennes globales affichées sur les cartes de chaque module.	Les moyennes pondérées sont correctes (arrondies au 0.5).	OK	REQ-F-003
ACC-02	Indicateurs	Tendances (Trends)	Vérifier les flèches (Up, Down, Stable) à côté des moyennes.	Une flèche verte, rouge ou orange s'affiche selon la tendance des dernières notes.	OK	
ACC-03	Navigation	Vue détaillée dynamique	Cliquer sur une carte de module spécifique dans la liste.	Le panneau latéral droit (Moyennes par branche + Graphique) et le panneau inférieur (Journal récent) s'ouvrent avec une animation fluide.	OK	
ACC-04	Navigation	Redimensionnement	Redimensionner la fenêtre de l'application (agrandir/réduire).	Les panneaux détaillés maintiennent leurs proportions (30% pour le panneau droit, 35% pour le journal).	OK	

4.7 Système d'exploitation (OS)

ID Test	Catégorie	Action à réaliser (Scénario)	Résultat attendu	Statut (OK/KO)	Remarques
OS-01	Application	Lancer l'application sous les trois OS ciblé (Windows, MacOS et Linux)	L'application fonctionne de manière inchangé	OK (Win x64, Linux x64, MacOS ARM64)	Manque de tests sur macOS x64, Win/Linux ARM64.
OS-02	Explorateur	Changer de BDD	L'explorateur de fichier doit s'ouvrir et passer le chemin du fichier.	OK	

5. Déploiement de l'application

Ce chapitre détaille les procédures permettant de compiler l'application de manière autonome (Self-Contained) et de créer les paquets d'installation pour les différents systèmes d'exploitation (Windows, Linux, macOS).

Bien que le framework .NET permette de compiler les exécutables bruts pour n'importe quelle plateforme depuis n'importe quel système d'exploitation (*cross-compilation*), la création des installateurs finaux requiert des environnements spécifiques :

- Génération des binaires (dotnet publish) : Peut être réalisée sous Windows, macOS ou Linux.
- Installateur Windows (Inno Setup) : Doit être généré depuis un environnement Windows.
- Paquets Linux (.deb et Flatpak) : Doivent être générés depuis un environnement Linux (ou via WSL - *Windows Subsystem for Linux* sous Windows).

5.1 Génération des exécutables (dotnet publish)

Le framework .NET et Avalonia UI permettent de compiler le code source pour de multiples architectures matérielles. La compilation se fait en mode "Single File" et "Self-Contained".

L'exécutable embarque ainsi son propre runtime .NET 10.0 ainsi que toutes ses dépendances natives, ce qui évite à l'utilisateur final d'avoir à installer des prérequis.

Afin d'optimiser les performances, le processus de publication utilise la suppression du code inutilisé (PublishTrimmed) et la précompilation (PublishReadyToRun).

Un script PowerShell (DotnetPublish.ps1) à été créer pour automatiser le processus suivant pour générer les builds de chaque plateforme :

5.1.1 Windows

```
# x86
dotnet publish -c Release -f net10.0 -r win-x86 --self-contained true `
  -p:PublishSingleFile=true `
  -p:PublishTrimmed=true `
  -p:PublishReadyToRun=true `
  -p:IncludeNativeLibrariesForSelfExtract=true `
  -p:IncludeAllContentForSelfExtract=true `
  -o "C:\Users\usr\Desktop\Build\WIN_x86"

# x64
dotnet publish -c Release -f net10.0 -r win-x64 --self-contained true `
  -p:PublishSingleFile=true `
  -p:PublishTrimmed=true `
  -p:PublishReadyToRun=true `
  -p:IncludeNativeLibrariesForSelfExtract=true `
  -p:IncludeAllContentForSelfExtract=true `
  -o "C:\Users\usr\Desktop\Build\WIN_x64"

# ARM64
dotnet publish -c Release -f net10.0 -r win-arm64 --self-contained true `
  -p:PublishSingleFile=true `
  -p:PublishTrimmed=true `
  -p:PublishReadyToRun=true `
  -p:IncludeNativeLibrariesForSelfExtract=true `
  -p:IncludeAllContentForSelfExtract=true `
  -o "C:\Users\usr\Desktop\Build\WIN_arm64"
```

5.1.2 Linux

```
# x64
dotnet publish -c Release -f net10.0 -r linux-x64 --self-contained true `
-p:PublishSingleFile=true `
-p:PublishTrimmed=true `
-p:PublishReadyToRun=true `
-p:IncludeNativeLibrariesForSelfExtract=true `
-p:IncludeAllContentForSelfExtract=true `
-o "C:\Users\usr\Desktop\Build\linux_x64"

# ARM64
dotnet publish -c Release -f net10.0 -r linux-arm64 --self-contained true `
-p:PublishSingleFile=true `
-p:PublishTrimmed=true `
-p:PublishReadyToRun=true `
-p:IncludeNativeLibrariesForSelfExtract=true `
-p:IncludeAllContentForSelfExtract=true `
-o "C:\Users\usr\Desktop\Build\linux_arm64"
```

5.1.3 MacOS

```
# x64
dotnet publish -c Release -f net10.0 -r osx-x64 --self-contained true `
-p:PublishSingleFile=true `
-p:PublishTrimmed=true `
-p:PublishReadyToRun=true `
-p:IncludeNativeLibrariesForSelfExtract=true `
-p:IncludeAllContentForSelfExtract=true `
-o "C:\Users\usr\Desktop\Build\osx_x64"

# ARM64
dotnet publish -c Release -f net10.0 -r osx-arm64 --self-contained true `
-p:PublishSingleFile=true `
-p:PublishTrimmed=true `
-p:PublishReadyToRun=true `
-p:IncludeNativeLibrariesForSelfExtract=true `
-p:IncludeAllContentForSelfExtract=true `
-o "C:\Users\usr\Desktop\Build\osx_arm64"
```

5.2 Création de l'installateur Windows Universel (Inno Setup)

Pour simplifier le déploiement sur Windows, un script Inno Setup (InnoSetupConfig.iss) a été développé pour créer un installateur "Universel". Cet installateur unique détecte automatiquement l'architecture de la machine cible au moment de l'installation et déploie le binaire approprié.

Le script vérifie l'architecture de l'OS grâce aux directives Check d'Inno Setup:

- Si le système est en 32-bit (Check: IsX86), il installe l'exécutable WIN_x86.
- Si le système est en 64-bit (Check: IsX64OS), il installe l'exécutable WIN_x64.
- Si le système est sur une architecture ARM (Check: IsArm64), il installe l'exécutable WIN_ARM64.

Le script se charge également de créer les raccourcis sur le bureau et dans le menu démarrer , et associe l'icône officielle de l'application.

5.3 Création du paquet Debian (.deb) pour Linux

La distribution sur les systèmes basés sur Debian (Ubuntu, Mint, etc.) nécessite la création d'un paquet .deb. La procédure est manuelle et doit être répétée pour chaque architecture (amd64, arm64).

Structure du dossier :

- Créer un dossier racine nommé selon l'architecture (ex: scholarlog-x64).
- Placer le binaire précompilé ScholarLog dans l'arborescence : scholarlog-x64/usr/bin/.
- Éditer le fichier de configuration DEBIAN/control pour y spécifier la version, l'architecture cible et le nom du mainteneur.
- Placer le fichier de raccourci scholarlog.desktop (spécifiant Categories=Education;Utility;) dans le dossier approprié (ex: /usr/share/applications/) pour l'intégration au menu système.

Compilation du paquet :

Avant de compiler, il est impératif d'attribuer les bons droits d'exécution et de lecture :

```
chmod 755 scholarlog-amd64/DEBIAN
chmod 644 scholarlog-amd64/DEBIAN/control
chmod 755 scholarlog-amd64
chmod 755 scholarlog-amd64/usr/bin/ScholarLog
```

Générer l'installateur avec la commande : `dpkg-deb --build scholarlog-amd64`.

5.4 Création du paquet Flatpak (Linux Universel)

Flatpak permet de distribuer l'application de manière sécurisée et universelle sur n'importe quelle distribution Linux, en isolant le logiciel dans une Sandbox.

Configuration de la Sandbox (Fichier .yml) :

Le manifeste com.HoferLukas.ScholarLog.yml autorise spécifiquement les accès matériels nécessaires à Avalonia UI :

- `--socket=x11` et `--socket=wayland` pour le serveur d'affichage.
- `--device=dri` (Direct Rendering Infrastructure) : indispensable pour l'accélération matérielle/GPU d'Avalonia.
- `--filesystem=host` : permet à l'application de lire et écrire la base de données locale SQLite sur l'ordinateur de l'étudiant.

Étapes de compilation :

1. Installer les prérequis : `sudo apt install -y flatpak flatpak-builder`.
2. Ajouter le dépôt Flathub et les SDK (Platform & Sdk en version 25.08).
3. Compiler le projet : `flatpak-builder build-dir com.HoferLukas.ScholarLog.yml --force-clean`.
4. Exporter vers un dépôt local : `flatpak build-export repo build-dir`.
5. Générer le bundle final .flatpak : `flatpak build-bundle repo ScholarLog-x64.flatpak com.HoferLukas.ScholarLog`.
6. L'application peut ensuite être installée par l'utilisateur avec : `flatpak install --user ScholarLog-x64.flatpak`. Le raccourci bureau sera alors automatiquement pris en charge par le fichier `com.HoferLukas.ScholarLog.desktop` intégré au bundle.

5.5 Script de déploiement totalement automatisé

Le script `BuildAndPackage.ps1` orchestre la création d'exécutables autonomes et de paquets d'installation pour Windows, Linux (Debian et Flatpak) et macOS.

Il utilise une approche hybride, combinant la puissance de .NET CLI sur Windows et l'environnement WSL (Windows Subsystem for Linux) pour générer les paquets natifs Linux.

Pour garantir l'exécution complète du script, l'environnement de développement doit être configuré comme suit :

Plateforme	Outils requis
Windows (Hôte)	.NET SDK 10.0, Inno Setup 6, PowerShell 5.1+
WSL (Linux)	dpkg, flatpak, flatpak-builder, qemu-user-static
Runtimes Linux	org.freedesktop.Platform (v25.08) et org.freedesktop.Sdk (v25.08) aarch64 et x64

Voici comment lancer le script : PS

Voici comment se comporte le script : `PS ...\scholarLog\src> .\BuildAndPackage.ps1`

1. Génération des exécutables (.NET Publish)
Compiler le code source C# en exécutables natifs "Self-Contained" (autonomes) pour sept architectures cibles (Windows x86/x64/ARM64, Linux x64/ARM64, macOS x64/ARM64).
2. Création des paquets Debian (.deb) : Le script structure l'arborescence (répertoires `usr/bin`, `DEBIAN`, etc.).

Crée des fichiers de contrôle (control, .desktop) en UTF-8 sans BOM.

Il force l'appartenance des fichiers au superutilisateur (root) afin que le paquet puisse être installé correctement sur une machine cible sans erreurs de permissions.

3. Création des paquets Flatpak : Produire une distribution universelle et sécurisée pour Linux via le système Flatpak.

Le script génère un manifeste YAML (app-id.yml) définissant les droits de la Sandbox (accès au réseau X11/Wayland, accélération matérielle dri pour Avalonia UI, et accès au système de fichiers host pour SQLite).

Il utilise flatpak-builder pour construire le bundle dans un environnement isolé, puis flatpak build-bundle pour exporter un fichier .flatpak installable.

4. Installation Windows (Inno Setup) : Créer un installateur .exe unique pour Windows.

Le script appelle l'exécutable ISCC.exe (compilateur Inno Setup) en lui passant le fichier de configuration .iss. L'installateur final est conçu pour détecter automatiquement l'architecture de l'utilisateur final et extraire le binaire approprié.

6. Licence du logiciel (GPLv3-only)

L'application ScholarLog est un logiciel libre, distribué exclusivement selon les termes de la licence GNU General Public License v3.0 uniquement (GPLv3-only).

La GPLv3 garantit que l'application demeurera un outil collaboratif, transparent et évolutif, tout en empêchant qu'une personne ou une entité ne s'approprie le code pour le rendre payant ou fermé.

La licence GPLv3 protège l'esprit open-source du projet à travers quatre principes fondamentaux :

- Liberté d'utilisation :
Exécution et utilisation de ScholarLog pour tout type d'usage, gratuitement et sans restriction.
- Accès et modification :
Accès complet au code source, permettant de l'étudier, de l'analyser et de le modifier pour l'adapter à des besoins spécifiques.
- Partage et redistribution : Autorisation et encouragement à redistribuer des copies conformes du logiciel auprès de pairs ou d'autres étudiants.
- Réciprocité (Copyleft strict) :
Obligation légale, en cas de distribution d'une version modifiée de ScholarLog, de la publier sous la même licence (GPLv3) et d'en rendre le code source public.

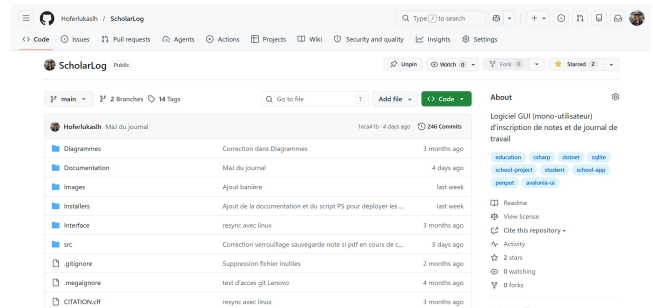
7. Gestionnaire de version

Tout au long du développement de l'application ScholarLog, un système de contrôle de version a été rigoureusement utilisé afin d'assurer un suivi précis des modifications et de sécuriser l'avancement du projet.

7.1 Git et GitHub

Le projet s'appuie sur le gestionnaire de version décentralisé Git, couplé à la plateforme d'hébergement GitHub.

Cette approche a offert plusieurs avantages majeurs :



- Traçabilité et débogage :
L'historique des *commits* a permis de comparer facilement la version de travail actuelle avec des versions antérieures. Cela s'est avéré indispensable pour isoler l'origine des bugs (comprendre pourquoi une fonctionnalité "ne marche plus") et annuler des modifications problématiques en revenant rapidement à un état stable.
- Accessibilité et transparence :
En accord avec l'esprit open source du projet, l'intégralité du code source est hébergée sur un dépôt public, le rendant totalement accessible pour la consultation.
- Gestion des jalons (Releases) :
Des *Releases* officielles ont été générées et publiées régulièrement sur le dépôt GitHub tout au long du cycle de développement, permettant de figer le code et de distribuer les exécutables correspondants aux différentes étapes clés de l'application.
- Lien vers le dépôt GitHub :
<https://github.com/Hoferlukaslh/ScholarLog>

7.2 Outil d'interface : LazyGit

Afin d'optimiser le flux de travail et de simplifier l'utilisation quotidienne des commandes Git, l'outil LazyGit a été employé.

Cette interface utilisateur de terminal (TUI) a grandement facilité la gestion des opérations courantes : la préparation des fichiers (staging), la création des commits, la lecture rapide des diffs (différences de code entre les versions) et la navigation dans l'historique. L'utilisation de cet outil a rendu le processus de développement plus fluide, visuel, et moins sujet aux erreurs de syntaxe propres aux lignes de commande classiques.

8. Conclusion

Ce projet de développement du module M5 a permis la réalisation complète de l'application ScholarLog, répondant avec succès au besoin initial : fournir aux étudiants de l'École Supérieure Technique un outil fiable, intuitif et autonome pour le suivi de leur résultat de formation.

8.1 objectifs et choix techniques

L'ensemble des exigences fonctionnelles et non-fonctionnelles définies dans le cahier des charges a été implémenté. Le choix d'une architecture orientée Modèle-Vue-Modèle (MVVM) couplée au framework Avalonia UI s'est avéré particulièrement pertinent pour garantir une portabilité native sur Windows, Linux et macOS, sans faire de compromis sur l'expérience utilisateur ou les performances. L'utilisation d'une base de données locale SQLite embarquée assure l'indépendance de l'application tout en garantissant la sécurité et l'intégrité des données scolaires.

L'un des défis techniques majeurs résidait dans l'intégration de documents numérisés (épreuves et scans) sans saturer la base de données. La solution novatrice mise en œuvre, basée sur la compression WebP et l'archivage CBZ en mémoire via SkiaSharp, démontre une optimisation particulièrement efficace. Elle permet de réduire l'empreinte de stockage de plus de 80 % tout en maintenant une navigation fluide.

8.2 Déploiement et automatisation

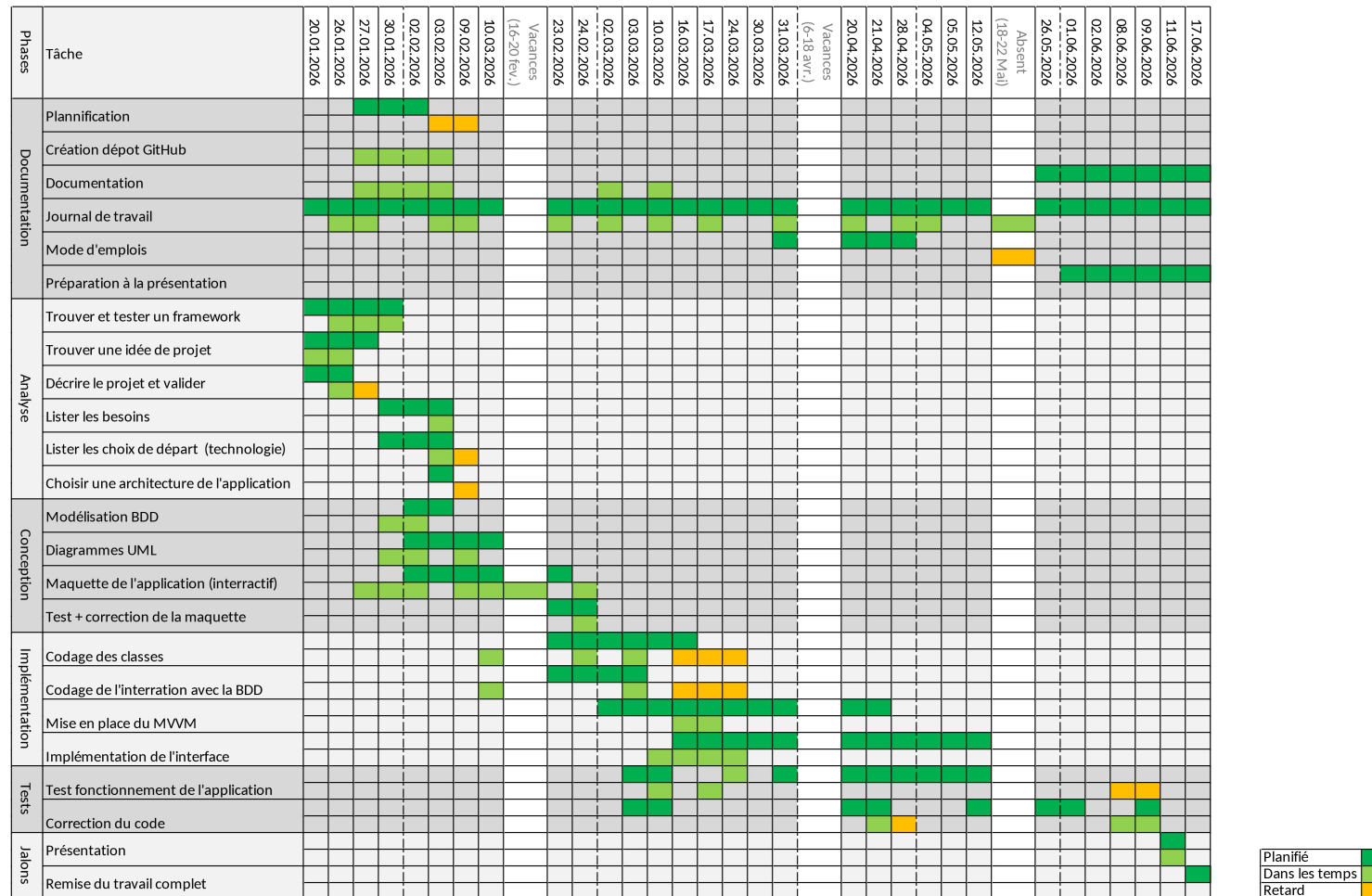
La mise en place d'un processus de déploiement totalement automatisé constitue un aboutissement professionnel majeur pour ce projet. La création de scripts PowerShell générant des exécutables autonomes, couplée à la création d'installateurs spécifiques (Inno Setup pour Windows, paquets Debian et Flatpak pour Linux), permet une distribution simplifiée du logiciel. ScholarLog est ainsi immédiatement accessible et installable pour l'utilisateur final, quel que soit son système d'exploitation.

8.3 Perspectives d'évolution

Bien que l'application soit pleinement fonctionnelle et prête à être déployée sous licence GPLv3, plusieurs axes d'amélioration peuvent être envisagés pour de futures versions :

- Amélioration de l'exportation :
Intégration des graphiques de répartition (Donut Charts) dans les exports. Par exemple en proposant un export en PDF.
- Personnalisation :
Implémentation d'un système de gestion de couleurs permettant aux utilisateurs de choisir leur jeu de couleurs (thèmes prédéfinis ou sélection libre).
- Validation matérielle :
Réalisation de tests poussés sur les architectures x64 de MacOS, Windows ARM64 et Linux ARM64 afin de certifier une compatibilité technique absolue.
- Affichage TM/PM (ou couleurs) pour être plus claire sur les valeurs afficher sur le dashboard.

9. Retour sur la planification



Globalement, la gestion du calendrier a été une réussite. Une excellente anticipation lors des phases de conception (UML, base de données et maquettage) a permis de démarrer certaines tâches en avance. Ce choix stratégique s'est avéré crucial pour absorber les imprévus techniques survenus lors de la phase d'implémentation.

10. Annexes et livrables

L'ensemble des documents liés à ce projet a été structuré et organisé afin de couvrir toutes les phases du développement, de la conception à la présentation finale. Voici la liste des livrables qui accompagnent cette documentation technique :

- LHO - M5 - Cahier des charges :
Définit les spécifications initiales, le public cible et les exigences du projet.
- LHO - M5 - DT :
Le présent document (Documentation Technique), détaillant l'architecture, la conception logicielle et les choix techniques.
- LHO - M5 - User Guide :
Manuel utilisateur rédigé en anglais, conçu dans une approche orientée client pour faciliter la prise en main de l'application.
- LHO - M5 - PoC Scans :
Preuve de concept (*Proof of Concept*) détaillant la stratégie technique d'optimisation et de compression (WebP/CBZ) pour la gestion des fichiers PDF joints aux évaluations.
- LHO - M5 - JournalPlanif :
Outil de suivi du temps de travail, incluant la planification initiale et la journalisation détaillée des tâches accomplies tout au long du projet.
- LHO - M5 - Présentation :
Support visuel utilisé lors de la présentation finale du logiciel.
- BDD_Exemple.db :
Fichier d'exemple de sauvegarde de l'application.
- ScholarLog_Installer_WIN_Universal.exe :
Installeur universel pour Windows (x86, x64, ARM64).
- ScholarLog_WIN_x64.exe :
Exécutable de l'application, sans installation.
- ScholarLog-main.zip :
Archive zip contenant tout le dépôt GitHub (en cas d'erreur d'accès ou de panne).
(Code source : dossier src)